claude-windows / c1fcb4aa-537d-41fd-858e-53f93349bf5a

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\subagents\workflows\wf_7049e1ed-b93\agent-a94589d078151a1a2.jsonl`
- SHA-256: `a7f5ec26fffeae88ff5af7a8e30f6e02d2877c8a2c747bd1098d8f2279ba42e5`
- Source modified: `2026-07-02T04:58:30+00:00`
- Imported at: `2026-07-05T16:48:24+00:00`
- project: `wf_7049e1ed-b93`
- session_id: `c1fcb4aa-537d-41fd-858e-53f93349bf5a`

Transcript

1. user (2026-07-02T04:55:36.183Z)

You are an adversarial code reviewer. Today is 2026-07-02. Review PR #417 in the repo at C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer (gh pr view 417, gh pr diff 417; the working copy is in worktree C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-cloudrun-cancel ? read code there for context).
The PR claims to fix: R1-20 (poll timeout abandons running GPU execution) (evidence in C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/AUDIT_REPORT_khelsutra-guru_2026-07-02.md ? Grep for the IDs; honor the verifier notes' scoping corrections).
Hunt REAL defects: correctness bugs in the diff, broken/vacuous tests, behavior changes beyond the brief, staging accidents (files that should not be in the diff ? check the file list!), convention violations (default-ON when brief says OFF, missing schema-version check, CWD assumptions), and whether the fix actually addresses the finding. READ-ONLY ? do not modify or comment on GitHub. blocker = must not merge; major = fix before merge; minor = nice-to-have.

2. user (2026-07-02T04:55:43.591Z)

```
title:  fix(compute): rescue-or-cancel on Cloud Run bucket-poll timeout (audit R1-21)
state:  OPEN
author: avidullu (Avi)
labels:
assignees:
reviewers:
projects:
milestone:
number: 417
url:    https://github.com/Khelsutra/badminton-highlight-indexer/pull/417
additions:      228
deletions:      8
auto-merge:     disabled
--
Fixes audit finding **R1-21** (AUDIT_REPORT_khelsutra-guru_2026-07-02): a bucket-poll timeout abandons a still-running L4 execution.
```

Summary

What: `CloudRunCompute.run_infer` blocked on `poll_until(...)`; when the poll deadline (`max_wait_sec`, default 1800s) hit, `PolicyTimeout` propagated straight to `main.py`'s dispatch handler, which marked the local job terminal-failed. Nothing cancelled the still-running Cloud Run execution (it kept billing the L4 until the job's `--task-timeout 3600`), and because the poll budget undercuts the task budget, a slow-but-healthy 30-60 min inference *systematically* ended as a local "cloud dispatch error" while its late done-marker + outputs were silently wasted.

How (on `PolicyTimeout`):
- **ONE final done-marker check** ? a worker that finished just past the poll deadline is rescued: its completed result is ingested normally (no cancel, no error). The final check is itself best-effort (a storage error there is treated as marker-absent, never masking the timeout).
- **Best-effort cancel** ? new `CloudRunJobClient.cancel_execution(execution)` abstract method (the injected test seam). The real `GoogleCloudRunJobClient` validates the stored execution name is fully-qualified (`.../executions/...`) **before** the lazy `google-cloud-run` import ? mirroring the #342 project guard so the guard is CI-exercisable ? then calls `run_v2.ExecutionsClient().cancel_execution`. (`run_job` falls back to the literal string `"execution"` when operation metadata carries no name; cancelling that would be a bogus API call, so it fails loudly instead.)
- **Honest failure message** ? the re-raised `PolicyTimeout` carries the job name, the execution name, and the cancel outcome ("cancel requested" / "best-effort cancel FAILED (?)") so the operator can verify the execution state in the GCP console. A cancel failure **never** masks the original timeout (exception type stays `PolicyTimeout`, chained `from` the original).

Behavior-preserving otherwise: the happy path (marker arrives in budget) is byte-identical; the timeout path previously raised `PolicyTimeout` and still does.

Not in this PR (deliberately): retuning `max_wait_sec` vs the deployed job's `--task-timeout` (a live-job policy change ? owner-gated per the M7 real-run convention), and the real-client cancel path itself is owner-verified on a real run (the SDK is intentionally not a CI dependency ? same stance as `run_job`).

Verification

Ran locally in the worktree (Windows, Python 3.13):
- `python -m pytest tests/test_compute_backend.py tests/test_api_cloud_dispatch.py -q` ? **42 passed** (includes 4 new tests: cancel called with the exact execution name; cancel failure still surfaces `PolicyTimeout` with both errors in the message; late done-marker rescues the result with zero cancels; real client rejects unresolvable execution names before the SDK import). Happy-path fakes now `raise AssertionError` from `cancel_execution`, locking in that cancel never fires outside the timeout path.
- `ruff check backend/compute/cloud_run.py tests/test_compute_backend.py tests/test_api_cloud_dispatch.py` ? clean.
- `mypy backend/compute/cloud_run.py` ? no new errors (the one reported error, `backend/storage/gcs.py:31 google.cloud has no attribute "storage"`, reproduces identically at base `e410136` with my change stashed ? a local-env artifact of partially-installed google namespace packages; CI's `mypy backend training` env resolves it via `ignore_missing_imports`).
- **Not verified locally:** the real `GoogleCloudRunJobClient.cancel_execution` against live GCP (no cloud spend from CI/dev; owner-verified on the next M7-style real run).

CI covers: full suite via `run_all_tests.py` (offline/mocked) + coverage floor, `ruff check .`, `mypy backend training`, link-check.

Checks
- [x] Targeted tests green; `ruff` clean on changed files; `mypy` no new errors; coverage not lowered (new branches are all test-covered). Full suite: CI.
- [x] No markdown changes ? link-check unaffected.

? Generated with [Claude Code](https://claude.com/claude-code)

3. user (2026-07-02T04:55:44.920Z)

213-
214-### R1-19 [P2/S] critical-fix ? badminton-highlight-indexer
215-**config.json output.output_dir is silently clobbered by the CLI --output-dir DEFAULT on every server/CLI start ? the typed knob is dead, and the DB path moves with it**
216-- anchor: `backend/main.py:1992` · tracked in: ? · finder: engine:correctness
217-- evidence: main.py:1837-1841 defines `--output-dir` with default='output'; main.py:1992 unconditionally does `global_config.output.output_dir = resolve_output_dir(args.output_dir)` ? the override fires even when the flag was never passed, so a config.json value can never take effect. models.py:274-276 documents the knob as if it's honored ('The CLI's --output-dir

```
overrides this at startup' ? implying only when given). db_path derives from it
(main.py:1995-1997).
218-- why it matters: A documented config knob that is a no-op on the main entry path, with
data-location consequences (perceived data loss / split state).
219-- proposed fix: Change argparse default to None and only override when args.output_dir is
not None (falling back to the config value through resolve_output_dir).
220-[Omitted long context line]
221-[Omitted long context line]
222-
223:### R1-20 [P2/S] critical-fix ? badminton-highlight-indexer
224-**upload_dir default 'output/uploads' is CWD-relative and never app-home-resolved ? packaged
--app writes uploads into an arbitrary CWD (or 500s on an unwritable one) while DB/reels live
under RALLY_APP_HOME**
225-- anchor: `backend/api/uploads.py:50` · tracked in: ? · finder: engine:correctness
226-- evidence: uploads.py:50 `upload_dir = getattr(sec, 'upload_dir', None) or
'output/uploads'`` used verbatim at :89 (os.makedirs), :95, :170 ? grep confirms no
resolve_output_dir/app_path is ever applied to it (only output.output_dir is resolved,
main.py:1992 via app_home.py:83-100). models.py:336 default 'output/uploads'.
227-- why it matters: Breaks or misplaces the primary ingest path of the packaged app; exactly
the class of CWD-vs-app-home bug P0a was built to eliminate (app_home.py:1-8).
228-- proposed fix: Resolve upload_dir through resolve_output_dir()/app_path() in _upload_cfg
(relative ? app-home-rooted; absolute passes through), mirroring output_dir.
229-[Omitted long context line]
230-[Omitted long context line]
231-
232-### R1-21 [P2/M] critical-fix ? badminton-highlight-indexer
233-**Cloud Run dispatch: a bucket-poll timeout abandons a still-running GPU execution ? no
cancellation, so the L4 keeps billing and its late result is never ingested**
```

4. user (2026-07-02T04:55:52.950Z)

```
diff --git a/backend/compute/cloud_run.py b/backend/compute/cloud_run.py
index ddfc085..598d62f 100644
--- a/backend/compute/cloud_run.py
+++ b/backend/compute/cloud_run.py
@@ -7,7 +7,10 @@
   2. **Bucket-poll** ``done_uri`` via ``execution_policy.poll_until`` + the storage layer (D16
? the
      completion signal is a tiny marker object the worker writes, NOT the Cloud Run API status,
so
      it is storage-backed and reuses the existing park/resume primitives). Scale-to-zero (D7):
the
-     cold-start is absorbed by the async poll.
+     cold-start is absorbed by the async poll. On a poll TIMEOUT (``max_wait_sec`` can undercut
the
+     job's own ``--task-timeout`` budget) the shim does ONE final marker check ? a slow-but-
healthy
+     worker that finished just past the deadline is rescued, not wasted ? then best-effort
CANCELS
+     the still-running execution so the L4 doesn't keep billing until ``--task-timeout``.
   3. **Read** the marker (it carries ``video_id`` + the worker's returncode) ? fetch
      ``outputs/<video_id>/report.json`` ? ``ComputeResult``.

@@ -33,6 +36,7 @@
 from backend.infrastructure.execution_policy import (
     DEFAULT_POLICY,
     ExecutionPolicy,
+    PolicyTimeout,
     poll_until,
 )
 from backend.storage import fetch, get_storage, parse_uri
@@ -65,6 +69,13 @@ def run_job(
         """Start a Cloud Run Job execution running ``python -m backend.worker <argv>``; return
an
         execution id/name (for logs). Does NOT block on completion (the caller bucket-
```

```
polls)."""

+    @abstractmethod
+    def cancel_execution(self, execution: str) -> None:
+        """Cancel a running execution by the name ``run_job`` returned. Called when the bucket-
poll
+        times out so the abandoned execution doesn't keep billing the GPU until ``--task-
timeout``.
+        May raise ? the caller treats every failure as best-effort (the original poll timeout
is
+        NEVER masked by a cancel error)."""
+

 class GoogleCloudRunJobClient(CloudRunJobClient):
     """Real client ? lazy-imports ``google-cloud-run``. Owner-verified on the M7 real run; CI
uses
@@ -111,6 +122,29 @@ def run_job(
             or "execution"
         )

+    def cancel_execution(self, execution: str) -> None:
+        # run_job falls back to the literal string "execution" when the operation metadata
carries
+        # no name ? that is NOT a resolvable resource name. Fail loudly BEFORE the SDK import
+        # (mirrors run_job's project guard, #342) so the misuse is exercisable in CI (where
+        # google-cloud-run is absent).
+        if "/executions/" not in (execution or ""):
+            raise RuntimeError(
+                f"not a fully-qualified Cloud Run execution name: {execution!r} ? expected "
+                "projects/<p>/locations/<l>/jobs/<j>/executions/<e> (run_job could not resolve
"
+                "one from the operation metadata, so there is nothing to cancel by name)."
+            )
+        try:
+            from google.cloud import run_v2  # type: ignore[attr-defined]
+        except Exception as exc:  # pragma: no cover - real SDK not present in CI
+            raise RuntimeError(
+                "google-cloud-run is required to cancel a Cloud Run execution; install it on
the "
+                "control plane (it is intentionally NOT a CI/test dependency)."
+            ) from exc
+        client = (
+            run_v2.ExecutionsClient()
+        )  # pragma: no cover - exercised only on a real run
+        client.cancel_execution(request=run_v2.CancelExecutionRequest(name=execution))
+

 class CloudRunCompute(ComputeBackend):
     """Dispatch to a Cloud Run GPU Job + bucket-poll for completion."""
@@ -170,12 +204,15 @@ def run_infer(self, spec: ComputeJobSpec) -> ComputeResult:
             done_uri,
         )

-        marker = poll_until(
-            lambda: self._read_marker(done_uri),
-            self._policy,
-            label="cloud-run-infer",
-            logger=LOG,
-        )
+        try:
+            marker = poll_until(
+                lambda: self._read_marker(done_uri),
+                self._policy,
+                label="cloud-run-infer",
+                logger=LOG,
```

```
+            )
+        except PolicyTimeout as timeout_exc:
+            marker = self._handle_poll_timeout(done_uri, str(execution), timeout_exc)
         video_id = str(marker.get("video_id", ""))
         # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
         # garbled marker must not masquerade as rc=0. (The worker always writes both fields.)
@@ -200,6 +237,53 @@ def run_infer(self, spec: ComputeJobSpec) -> ComputeResult:
             execution=str(execution or ""),
         )

+    def _handle_poll_timeout(
+        self, done_uri: str, execution: str, timeout_exc: PolicyTimeout
+    ) -> dict:
+        """Poll deadline hit ? rescue a just-finished result, else cancel + re-raise.
+
+        The poll budget (``max_wait_sec``, default 1800s) can undercut the deployed job's own
+        ``--task-timeout`` (3600s), so a slow-but-healthy worker may finish just past the
deadline:
+        ONE final marker check rescues that completed GPU run instead of wasting it. If the
marker
+        is genuinely absent, the execution is still running ? best-effort cancel it (otherwise
the
+        L4 keeps billing until ``--task-timeout``) and re-raise a ``PolicyTimeout`` carrying
the
+        execution name so the operator can verify the state in the GCP console. A cancel
failure
+        NEVER masks the original timeout."""
+        try:
+            late = self._read_marker(done_uri)
+        except Exception:  # noqa: BLE001 ? the final check is best-effort too: absence ?
failure
+            LOG.warning(
+                "cloud-run: final post-timeout marker check failed ? treating as absent",
+                exc_info=True,
+            )
+            late = None
+        if late is not None:
+            LOG.warning(
+                "cloud-run: job=%s exec=%s done-marker found on the final post-timeout check "
+                "? rescuing the completed result",
+                self._job_name,
+                execution,
+            )
+            return late
+        try:
+            self._client.cancel_execution(execution)
+        except Exception as cancel_exc:  # noqa: BLE001 ? best-effort: never mask the timeout
+            cancel_note = f"best-effort cancel FAILED ({cancel_exc})"
+            LOG.warning(
+                "cloud-run: best-effort cancel of execution=%s failed: %s",
+                execution,
+                cancel_exc,
+                exc_info=True,
+            )
+        else:
+            cancel_note = "cancel requested"
+            LOG.info("cloud-run: requested cancel of execution=%s", execution)
+        raise PolicyTimeout(
+            f"{timeout_exc} [cloud-run job={self._job_name} "
+            f"execution={execution or '<unknown>'}; {cancel_note} ? verify the execution "
+            "state in the GCP console]"
+        ) from timeout_exc
+
     # --- storage-backed completion signal (bucket-poll) ---
```

```
     def _read_marker(self, done_uri: str) -> Optional[dict]:
         """The done-marker dict if the worker has written it, else None ? poll_until keeps
waiting."""
diff --git a/tests/test_api_cloud_dispatch.py b/tests/test_api_cloud_dispatch.py
index 5e2e46f..53923a9 100644
--- a/tests/test_api_cloud_dispatch.py
+++ b/tests/test_api_cloud_dispatch.py
@@ -88,6 +88,11 @@ def run_job(self, job_name, argv, *, region, project=None):
             )
         return "exec-1"

+    def cancel_execution(self, execution):
+        raise AssertionError(
+            "cancel must never fire on the happy path"
+        )  # pragma: no cover
+

 @pytest.fixture
 def cloud_env(tmp_path, monkeypatch):
diff --git a/tests/test_compute_backend.py b/tests/test_compute_backend.py
index 2955192..61d82fe 100644
--- a/tests/test_compute_backend.py
+++ b/tests/test_compute_backend.py
@@ -15,7 +15,7 @@
 from backend.compute import ComputeJobSpec, get_compute_backend
 from backend.compute.cloud_run import CloudRunCompute, CloudRunJobClient
 from backend.config import AppConfig
-from backend.infrastructure.execution_policy import ExecutionPolicy
+from backend.infrastructure.execution_policy import ExecutionPolicy, PolicyTimeout

 # A fast, no-wait policy (the fake client writes the marker synchronously, so the first poll
hits).
 _FAST = ExecutionPolicy(poll_every_n_sec=0.0, op_timeout_sec=5.0, max_wait_sec=5.0)
@@ -63,6 +63,11 @@ def run_job(self, job_name, argv, *, region, project=None):
             )
         return "exec-123"

+    def cancel_execution(self, execution):
+        raise AssertionError(
+            "cancel must never fire on the happy path"
+        )  # pragma: no cover
+

 class _NoopRunClient(CloudRunJobClient):
     """Records the dispatch but does NO filesystem writes ? for tests that fake the storage
layer
@@ -70,6 +75,7 @@ class _NoopRunClient(CloudRunJobClient):

     def __init__(self):
         self.calls: list[dict] = []
+        self.cancelled: list[str] = []

     def run_job(self, job_name, argv, *, region, project=None):
         self.calls.append(
@@ -77,6 +83,32 @@ def run_job(self, job_name, argv, *, region, project=None):
         )
         return "exec"

+    def cancel_execution(self, execution):
+        self.cancelled.append(execution)
+
+
+class _NeverDoneClient(CloudRunJobClient):
+    """Simulates a still-running execution: dispatch succeeds (returning a fully-qualified
+    execution name) but NO done-marker is ever written, so the bucket-poll must time out."""
```

```
+
+    EXEC_NAME = "projects/p/locations/asia-south1/jobs/rally-worker/executions/exec-slow"
+
+    def __init__(self, *, cancel_error: Exception | None = None):
+        self.calls: list[dict] = []
+        self.cancelled: list[str] = []
+        self._cancel_error = cancel_error
+
+    def run_job(self, job_name, argv, *, region, project=None):
+        self.calls.append(
+            {"job": job_name, "argv": list(argv), "region": region, "project": project}
+        )
+        return self.EXEC_NAME
+
+    def cancel_execution(self, execution):
+        self.cancelled.append(execution)
+        if self._cancel_error is not None:
+            raise self._cancel_error
+

 # ------------------------------------------------------------------ factory (default-OFF)

@@ -120,6 +152,19 @@ def test_real_client_rejects_missing_project():
             )


+def test_real_client_cancel_rejects_unresolvable_execution_name():
+    """cancel_execution must fail loudly on a non-resolvable execution name ? run_job falls
back
+    to the literal "execution" when the operation metadata carries no name, and cancelling that
+    would be a bogus API call. The guard runs BEFORE the google-cloud-run import (mirrors the
+    project guard), so it is exercisable in CI (the SDK is intentionally not a test
dependency)."""
+    from backend.compute.cloud_run import GoogleCloudRunJobClient
+
+    client = GoogleCloudRunJobClient()
+    for bogus in ("", "execution", "exec-123"):
+        with pytest.raises(RuntimeError, match="fully-qualified"):
+            client.cancel_execution(bogus)
+
+
 # ------------------------------------------------------------- CloudRunCompute dispatch + poll


@@ -267,6 +312,92 @@ def fake_fetch(uri, dst=None, *, config=None, overwrite=False):
     )  # >=1 absent poll occurred before the marker appeared (loop ran)


+# ---------------------------------------- poll timeout: rescue-or-cancel (R1-21 GPU cost
leak)
+
+# max_wait_sec=0.0 ? poll_until gives up after its FIRST absent check (no real waiting).
+_TIMEOUT_NOW = ExecutionPolicy(poll_every_n_sec=0.0, op_timeout_sec=5.0, max_wait_sec=0.0)
+
+
+def test_poll_timeout_cancels_execution_and_raises(tmp_path):
+    """Poll deadline + no marker ? the still-running execution is cancelled (with the EXACT
name
+    run_job returned) and PolicyTimeout surfaces carrying that name for the operator."""
+    client = _NeverDoneClient()
+    backend = CloudRunCompute(
+        run_client=client,
+        job_name="rally-worker",
+        region="asia-south1",
+        policy=_TIMEOUT_NOW,
```

```
+        token="tok",
+    )
+    with pytest.raises(PolicyTimeout) as exc:
+        backend.run_infer(
+            ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bucket"))
+        )
+    assert client.cancelled == [_NeverDoneClient.EXEC_NAME]  # cancel got the right name
+    msg = str(exc.value)
+    assert _NeverDoneClient.EXEC_NAME in msg  # operator can find the execution in GCP
+    assert "cancel requested" in msg
+    assert "max_wait_sec" in msg  # the original timeout text is preserved, not masked
+
+
+def test_poll_timeout_cancel_failure_still_raises_policy_timeout(tmp_path):
+    """Best-effort means BEST-EFFORT: a cancel API failure must never mask the original timeout
?
+    the caller (main.py's dispatch handler) still sees PolicyTimeout, with the failure
noted."""
+    client = _NeverDoneClient(cancel_error=RuntimeError("cancel API unreachable"))
+    backend = CloudRunCompute(
+        run_client=client,
+        job_name="rally-worker",
+        region="asia-south1",
+        policy=_TIMEOUT_NOW,
+        token="tok",
+    )
+    with pytest.raises(PolicyTimeout) as exc:
+        backend.run_infer(
+            ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bucket"))
+        )
+    assert client.cancelled == [_NeverDoneClient.EXEC_NAME]  # the attempt WAS made
+    msg = str(exc.value)
+    assert _NeverDoneClient.EXEC_NAME in msg
+    assert "cancel API unreachable" in msg  # the cancel failure is surfaced, not swallowed
+    assert "max_wait_sec" in msg  # ...alongside the original timeout text
+
+
+def test_poll_timeout_late_marker_rescues_result(monkeypatch):
+    """A slow-but-healthy worker that finishes JUST past the poll deadline (max_wait_sec=1800
can
+    undercut the job's --task-timeout=3600) is rescued by the ONE final post-timeout marker
check:
+    the completed GPU result is ingested, nothing is cancelled, no timeout surfaces."""
+    from backend.compute import cloud_run as cr
+
+    exists_calls = {"n": 0}
+
+    class _FakeStore:
+        def exists(self, ref):
+            exists_calls["n"] += 1
+            return exists_calls["n"] >= 2  # absent for the poll, present on the final check
+
+    monkeypatch.setattr(cr, "get_storage", lambda backend, config=None: _FakeStore())
+
+    def fake_fetch(uri, dst=None, *, config=None, overwrite=False):
+        if dst is None:
+            dst = os.path.join(tempfile.mkdtemp(prefix="cr-late-"), "m.json")
+        with open(dst, "w", encoding="utf-8") as f:
+            f.write('{"video_id": "vlate", "returncode": 0, "segment_count": 1}')
+        return dst
+
+    monkeypatch.setattr(cr, "fetch", fake_fetch)
+    client = _NoopRunClient()
+    backend = cr.CloudRunCompute(
+        run_client=client, job_name="j", region="r", token="t", policy=_TIMEOUT_NOW
```

```
+    )
+    result = backend.run_infer(
+        ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri="gs://b")
+    )
+    assert result.returncode == 0
+    assert result.video_id == "vlate"  # the late result is ingested, not wasted
+    assert client.cancelled == []  # a finished execution is never cancelled
+    assert exists_calls["n"] == 2  # exactly ONE extra post-timeout check
+
+
 # ------------------------------------------------- worker wiring: dispatch branch + done-
marker
```

5. user (2026-07-02T04:55:55.056Z)

```
232:### R1-21 [P2/M] critical-fix ? badminton-highlight-indexer
233-**Cloud Run dispatch: a bucket-poll timeout abandons a still-running GPU execution ? no
cancellation, so the L4 keeps billing and its late result is never ingested**
234-- anchor: `backend/compute/cloud_run.py:173` · tracked in: ? · finder: engine:correctness
235-- evidence: cloud_run.py:173-178: run_infer blocks on poll_until(...);
execution_policy.py:183-189 raises PolicyTimeout after max_wait_sec (default 1800s). The
exception propagates to main.py:1068-1070 which marks the job failed (`cloud dispatch error`) ?
no code anywhere in cloud_run.py calls executions.cancel/delete (grep: no cancel API), and the
late-arriving done-marker/intervals.csv are never ingested (the local job is already terminal).
236-- why it matters: Direct GPU cost leak plus wasted results on the cloud-serving path (owner-
gated go-live area, but the code path is live behind compute='cloud').
237-- proposed fix: On PolicyTimeout, best-effort cancel the execution via the Jobs API
(executions.cancel with the stored execution name) and log the execution id in the failure
message; optionally check the done-marker once more before failing.
238-[Omitted long context line]
239-[Omitted long context line]
240-
241-### R1-22 [P2/M] tech-debt ? badminton-highlight-indexer
242-**Stage-2 AI-handoff orchestration duplicated between the default segmenter (yolo_hybrid)
and the trajectory family**
243-- anchor: `backend/pipeline/segmenters/trajectory_hybrid.py:390` · tracked in: ? · finder:
engine:structure
244-[Omitted long context line]
245-- why it matters:
246-- proposed fix: Extract a shared handoff helper (e.g.
backend/pipeline/segmenters/handoff.py: pad_window + process_candidate_window + the executor
fan-out) consumed by both HybridYoloSegmenter and TrajectoryHybridSegmenter; move _fmt_ts to
backend/utils. Note the trajectory subclass family itself (fusion/tracknet/wasb via
_build_runner template methods) is well-factored ? the duplication is only across the
yolo/trajectory family boundary.
247-[Omitted long context line]
248-[Omitted long context line]
249-
250-### R1-23 [P2/L] tech-debt ? badminton-highlight-indexer
251-**Four more god modules over 800 lines; database.py is a 50-method monolith**
252-- anchor: `backend/infrastructure/database.py:25` · tracked in: mypy.ini comments reference
the typed-config migration (D3) in docs/CODE_CLEANUP_AUDIT_2026-06-24.md · finder:
engine:structure
```

6. user (2026-07-02T04:56:03.003Z)

```
228     - proposed fix: Resolve upload_dir through resolve_output_dir()/app_path() in
_upload_cfg (relative ? app-home-rooted; absolute passes through), mirroring output_dir.
229     - verifier notes: Sharper failure scenario: on macOS the .command runs with Terminal's
CWD=$HOME, and any `indexer --app` shell launch (advertised in install.py:243) uses the caller's
CWD ? uploads land in e.g. ~/output/uploads while DB/reels live under ~/Library/Application
Support/Khelsutra/output; unwritable CWD ? PermissionError at uploads.py:89 ? 500 on
/api/upload/init. Caveat: Windows double-click of KhelSutra.bat coincidentally works (the .bat
lives in app_home, install.py:175, and cmd sets CWD to the .bat dir). Extra impact the finder
missed: tools/cleanup_caches.py:403 sweeps <output-dir>/uploads (app-home-rooted post-P0a) and
```

the uninstall manifest user_data (install.py:117-121) lists only app-home paths, so stray multi-GB uploads are never swept and survive uninstall --purge. Fix as proposed is sane: resolve_output_dir() in _upload_cfg is byte-identical with RALLY_APP_HOME unset (app_home.py:94-95) and existing tests use absolute tmp paths (test_upload_download.py:31) that pass through.
230   - verifier reasoning: uploads.py:50 `upload_dir = getattr(sec, "upload_dir", None) or "output/uploads"` used verbatim at :89 os.makedirs, :93, :95, :160, :170 ? no resolver applied (repo-wide grep). main.py:1992 resolves ONLY output.output_dir via resolve_output_dir (app_home.py:83-100); models.py:336 default is relative "output/uploads"; the packaged launcher (install.py:81-99) sets RALLY_APP_HOME but never cd's, and the first-run seeded config (loader.py:261-265 LIGHT_DEFAULT_CONFIG) has no security block, so the relative default is live in every packaged install. git log -- backend/api/uploads.py (top f28af73) shows no fix. Not owner-gated (local packaging correctness).
231
232   ### R1-21 [P2/M] critical-fix ? badminton-highlight-indexer
233   **Cloud Run dispatch: a bucket-poll timeout abandons a still-running GPU execution ? no cancellation, so the L4 keeps billing and its late result is never ingested**
234   - anchor: `backend/compute/cloud_run.py:173` · tracked in: ? · finder: engine:correctness
235   - evidence: cloud_run.py:173-178: run_infer blocks on poll_until(...); execution_policy.py:183-189 raises PolicyTimeout after max_wait_sec (default 1800s). The exception propagates to main.py:1068-1070 which marks the job failed (`cloud dispatch error`) ? no code anywhere in cloud_run.py calls executions.cancel/delete (grep: no cancel API), and the late-arriving done-marker/intervals.csv are never ingested (the local job is already terminal).
236   - why it matters: Direct GPU cost leak plus wasted results on the cloud-serving path (owner-gated go-live area, but the code path is live behind compute='cloud').
237   - proposed fix: On PolicyTimeout, best-effort cancel the execution via the Jobs API (executions.cancel with the stored execution name) and log the execution id in the failure message; optionally check the done-marker once more before failing.
238   - verifier notes: Correction to the failure scenario: "the L4 keeps billing" is bounded, not open-ended ? the job is created with --max-retries 0 --task-timeout 3600 (deploy/cloudrun/README.md:33), so the orphaned execution bills at most ~30 extra minutes past the 1800s poll deadline, then Cloud Run kills it. The sharper, systematic scenario is the policy/task-timeout mismatch (1800s poll vs 3600s task budget): a long match video whose GPU inference takes 30-60 min ALWAYS ends as a local "cloud dispatch error" failure while the cloud run later succeeds, writes the done-marker + outputs to the bucket, and the result is silently wasted ? user pays full GPU cost and must re-dispatch. Fix scoping corrections: (a) cancellation must be a new abstract method on CloudRunJobClient (the injected test seam) with the real impl using run_v2.ExecutionsClient().cancel_execution, owner-verified like run_job since the SDK is not a CI dep; (b) it must be best-effort ? run_job's return falls back to the literal string "execution" when op.metadata.name is absent (cloud_run.py:108-112), which is not a resolvable execution name; (c) the "check the marker once more before failing" suggestion is good and cheap; also consider aligning max_wait_sec with the job's task-timeout. Owner-gating: the shim fix + fake-client tests are doable now; only the real-client cancel verification and any policy retune on the live job are owner-gated (M7 real-run / real cloud spend convention), which matches the finding's own caveat.
239   - verifier reasoning: Personally reproduced from current code (post-pull, HEAD e410136). (1) backend/compute/cloud_run.py:173-178 ? run_infer blocks on poll_until(self._read_marker(done_uri), self._policy) with no exception handling; backend/infrastructure/execution_policy.py:54 sets max_wait_sec=1800.0 default and :183-189 raises PolicyTimeout at the deadline. (2) backend/main.py:1064-1070 catches the exception and returns _failed(f"cloud dispatch error: {e}", True) ? the local job goes terminal-failed, so the late done-marker/outputs are never ingested. (3) No cancellation exists anywhere: grep for cancel/executions./CancelExecution across backend/*.py hits only the minified UI asset (backend/ui/assets/index-ukuS9wxa.js); GoogleCloudRunJobClient (cloud_run.py:69-112) only implements run_job, and the CloudRunJobClient ABC (:50-66) has no cancel method. (4) Not already fixed: git log -15 -- backend/compute/cloud_run.py shows only formatting (b6214e5), isort (f523d3a), an unrelated project-guard fix (688d28f), and the original M6/provenance commits. (5) The mechanism is worse than a rare edge: get_compute_backend (backend/compute/__init__.py:39,76) passes DEFAULT_POLICY (1800s) while the deployed job uses --task-timeout 3600 (deploy/cloudrun/README.md:33, docs/CONVERT_GCS_VIDEOS_TO_HIGHLIGHTS.md:57), so any execution legitimately taking 30-60 min is systematically abandoned at 30 min, keeps billing the L4 for up to another 30 min, then its completed result is dropped. deploy/cloudrun/README.md:60-64 documents the timeout path but only for hung/crashed containers ? the slow-but-healthy case was not considered.

```
240
241     ### R1-22 [P2/M] tech-debt ? badminton-highlight-indexer
242     **Stage-2 AI-handoff orchestration duplicated between the default segmenter
(yolo_hybrid) and the trajectory family**
243     - anchor: `backend/pipeline/segmenters/trajectory_hybrid.py:390` · tracked in: ? ·
finder: engine:structure
```

7. user (2026-07-02T04:56:05.764Z)

```
1d94e2b fix(compute): rescue-or-cancel on Cloud Run bucket-poll timeout (R1-21)
e410136 api: /api/videos surfaces fps + resolution (from the stored ffprobe details) (#413)
81a78bf api: /api/videos returns video_id + a friendly match_name (fix disappearing name) (#412)
8b16cf4 docs: reflect today's storage endpoints + the corpus/user-video boundary (#411)
aa915a7 deploy/gcp: centralize project + bucket names in one sourced buckets.env (#410)
backend/compute/cloud_run.py
tests/test_api_cloud_dispatch.py
tests/test_compute_backend.py
```

8. assistant (2026-07-02T04:56:55.389Z)

```
The PR file list is clean (3 files, all in scope). Now let me read the changed code in the
worktree for context.
```

9. user (2026-07-02T04:56:56.216Z)

```
1       """``CloudRunCompute`` ? dispatch one inference job to a Cloud Run GPU **Job** and
bucket-poll
2       for its completion (COMPUTE_DECOUPLED_SERVING M6, decisions D16 + D7).
3
4       Flow (the cloud-serving compute path):
5         1. **Dispatch** a Cloud Run Job execution whose container runs ``python -m
backend.worker infer
6            --video-uri <gs> --out-uri <gs> --done-uri <gs> ?`` (the SAME worker, on the L4
GPU).
7         2. **Bucket-poll** ``done_uri`` via ``execution_policy.poll_until`` + the storage
layer (D16 ? the
8            completion signal is a tiny marker object the worker writes, NOT the Cloud Run API
status, so
9            it is storage-backed and reuses the existing park/resume primitives). Scale-to-zero
(D7): the
10           cold-start is absorbed by the async poll. On a poll TIMEOUT (``max_wait_sec`` can
undercut the
11           job's own ``--task-timeout`` budget) the shim does ONE final marker check ? a slow-
but-healthy
12           worker that finished just past the deadline is rescued, not wasted ? then best-
effort CANCELS
13           the still-running execution so the L4 doesn't keep billing until ``--task-
timeout``.
14        3. **Read** the marker (it carries ``video_id`` + the worker's returncode) ? fetch
15           ``outputs/<video_id>/report.json`` ? ``ComputeResult``.
16
17      The dispatched container is forced to run **INLINE** (``compute_target=local_gpu`` +
native detector)
18      so it never recursively re-dispatches ? the dispatcher decides cloud, the container just
computes.
19
20      The Cloud Run trigger is an **injected client** (``CloudRunJobClient``) so the whole
shim is unit-tested
21      with a fake (no GCP). The real ``GoogleCloudRunJobClient`` lazy-imports ``google-cloud-
run`` and is
22      owner-verified on the M7 real run.
23      """
24
25      from __future__ import annotations
```

```

    import json
    import logging
    import os
    import tempfile
    import uuid
    from abc import ABC, abstractmethod
    from typing import Any, List, Optional

    from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
    from backend.infrastructure.execution_policy import (
        DEFAULT_POLICY,
        ExecutionPolicy,
        PolicyTimeout,
        poll_until,
    )
    from backend.storage import fetch, get_storage, parse_uri

    LOG = logging.getLogger("compute.cloud_run")

    # The container worker must run INLINE on the GPU ? never re-enter the cloud-dispatch
branch.
    # (compute_target=local_gpu disarms the cmd_infer dispatch guard; native = the L4 WASB
runner.)
    _DEFAULT_CONTAINER_OVERRIDES: tuple[str, ...] = (
        "deployment.compute_target=local_gpu",
        "indexing.detector_impl=native",
    )


    class CloudRunJobClient(ABC):
        """Triggers a Cloud Run **Job** execution with per-execution container arg
overrides.

        Abstracted so ``CloudRunCompute`` is testable with a fake (no GCP). A real
implementation
        overrides the job's container ``args`` for this execution and starts it."""

        @abstractmethod
        def run_job(
            self,
            job_name: str,
            argv: List[str],
            *,
            region: str,
            project: Optional[str] = None,
        ) -> str:
            """Start a Cloud Run Job execution running ``python -m backend.worker <argv>``;
return an
            execution id/name (for logs). Does NOT block on completion (the caller bucket-
polls)."""

        @abstractmethod
        def cancel_execution(self, execution: str) -> None:
            """Cancel a running execution by the name ``run_job`` returned. Called when the
bucket-poll
            times out so the abandoned execution doesn't keep billing the GPU until
``--task-timeout``.
            May raise ? the caller treats every failure as best-effort (the original poll
timeout is
            NEVER masked by a cancel error)."""


    class GoogleCloudRunJobClient(CloudRunJobClient):
        """Real client ? lazy-imports ``google-cloud-run``. Owner-verified on the M7 real
```

```
run; CI uses
82          a fake, so this code path is never exercised in tests (the SDK isn't a test
dependency)."""
83
84          def run_job(
85              self,
86              job_name: str,
87              argv: List[str],
88              *,
89              region: str,
90              project: Optional[str] = None,
91          ) -> str:
92              # A None/empty project would build a bogus "projects/None/locations/..." path
(job_path
93              # requires a real str). Fail loudly BEFORE the SDK import so the misconfig is
exercisable
94              # in CI (where google-cloud-run is absent) and so mypy narrows project to str
for job_path.
95              if not project:
96                  raise RuntimeError(
97                      "GOOGLE_CLOUD_PROJECT is unset ? the real Cloud Run dispatch needs a GCP
project id "
98                      "to resolve the job path; set it on the control plane (see
get_compute_backend)."
99                  )
100             try:
101                 from google.cloud import run_v2  # type: ignore[attr-defined]
102             except Exception as exc:  # pragma: no cover - real SDK not present in CI
103                 raise RuntimeError(
104                     "google-cloud-run is required for CloudRunCompute's real client; install
it on the "
105                     "control plane (it is intentionally NOT a CI/test dependency)."
106                 ) from exc
107             client = (
108                 run_v2.JobsClient()
109             )  # pragma: no cover - exercised only on the real M7 run
110             name = client.job_path(project, region, job_name)
111             overrides = run_v2.RunJobRequest.Overrides(
112                 container_overrides=[
113                     run_v2.RunJobRequest.Overrides.ContainerOverride(args=argv)
114                 ]
115             )
116             op = client.run_job(
117                 request=run_v2.RunJobRequest(name=name, overrides=overrides)
118             )
119             return (
120                 getattr(op, "metadata", None)
121                 and getattr(op.metadata, "name", "")
122                 or "execution"
123             )
124
125         def cancel_execution(self, execution: str) -> None:
126             # run_job falls back to the literal string "execution" when the operation
metadata carries
127             # no name ? that is NOT a resolvable resource name. Fail loudly BEFORE the SDK
import
128             # (mirrors run_job's project guard, #342) so the misuse is exercisable in CI
(where
129             # google-cloud-run is absent).
130             if "/executions/" not in (execution or ""):
131                 raise RuntimeError(
132                     f"not a fully-qualified Cloud Run execution name: {execution!r} ?
expected "
133                     "projects/<p>/locations/<l>/jobs/<j>/executions/<e> (run_job could not
resolve "
```

```
                    "one from the operation metadata, so there is nothing to cancel by
name)."
                )
            try:
                from google.cloud import run_v2  # type: ignore[attr-defined]
            except Exception as exc:  # pragma: no cover - real SDK not present in CI
                raise RuntimeError(
                    "google-cloud-run is required to cancel a Cloud Run execution; install
it on the "
                    "control plane (it is intentionally NOT a CI/test dependency)."
                ) from exc
            client = (
                run_v2.ExecutionsClient()
            )  # pragma: no cover - exercised only on a real run
            client.cancel_execution(request=run_v2.CancelExecutionRequest(name=execution))


    class CloudRunCompute(ComputeBackend):
        """Dispatch to a Cloud Run GPU Job + bucket-poll for completion."""

        def __init__(
            self,
            *,
            run_client: CloudRunJobClient,
            job_name: str,
            region: str,
            project: Optional[str] = None,
            storage_config: Optional[dict] = None,
            policy: ExecutionPolicy = DEFAULT_POLICY,
            token: Optional[str] = None,
            container_overrides: Optional[tuple[str, ...]] = None,
        ) -> None:
            self._client = run_client
            self._job_name = job_name
            self._region = region
            self._project = project
            self._storage_config = storage_config or {}
            self._policy = policy
            # The dispatch token keys the done-marker so concurrent jobs to one bucket don't
collide.
            # None ? a fresh uuid4 per run_infer call (prod); injected ? deterministic
(tests).
            self._token = token
            self._container_overrides = (
                _DEFAULT_CONTAINER_OVERRIDES
                if container_overrides is None
                else tuple(container_overrides)
            )

        def run_infer(self, spec: ComputeJobSpec) -> ComputeResult:
            out_root = spec.out_uri.rstrip("/")
            token = self._token or uuid.uuid4().hex
            done_uri = f"{out_root}/_dispatch/{token}.done"
            argv: List[str] = [
                "infer",
                "--video-uri",
                spec.video_uri,
                "--out-uri",
                spec.out_uri,
                "--done-uri",
                done_uri,
            ]
            if spec.segmenter:
                argv += ["--segmenter", spec.segmenter]
            for kv in (*spec.config_overrides, *self._container_overrides):
```

```
                argv += ["--set", kv]

            execution = self._client.run_job(
                self._job_name, argv, region=self._region, project=self._project
            )
            LOG.info(
                "cloud-run: dispatched job=%s exec=%s; bucket-polling %s",
                self._job_name,
                execution,
                done_uri,
            )

            try:
                marker = poll_until(
                    lambda: self._read_marker(done_uri),
                    self._policy,
                    label="cloud-run-infer",
                    logger=LOG,
                )
            except PolicyTimeout as timeout_exc:
                marker = self._handle_poll_timeout(done_uri, str(execution), timeout_exc)
            video_id = str(marker.get("video_id", ""))
            # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
            # garbled marker must not masquerade as rc=0. (The worker always writes both
fields.)
            rc = int(marker.get("returncode", 1))
            if not video_id:
                LOG.warning(
                    "cloud-run: job=%s completion marker present but empty/unreadable (no "
                    "video_id) ? treating as failure",
                    self._job_name,
                )
                rc = rc or 1
            outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
            report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
            LOG.info(
                "cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id, rc
            )
            return ComputeResult(
                returncode=rc,
                outputs_uri=outputs_uri,
                video_id=video_id,
                report=report,
                execution=str(execution or ""),
            )

        def _handle_poll_timeout(
            self, done_uri: str, execution: str, timeout_exc: PolicyTimeout
        ) -> dict:
            """Poll deadline hit ? rescue a just-finished result, else cancel + re-raise.

            The poll budget (``max_wait_sec``, default 1800s) can undercut the deployed
job's own
            ``--task-timeout`` (3600s), so a slow-but-healthy worker may finish just past
the deadline:
            ONE final marker check rescues that completed GPU run instead of wasting it. If
the marker
            is genuinely absent, the execution is still running ? best-effort cancel it
(otherwise the
            L4 keeps billing until ``--task-timeout``) and re-raise a ``PolicyTimeout``
carrying the
            execution name so the operator can verify the state in the GCP console. A cancel
failure
            NEVER masks the original timeout."""
```

```
252             try:
253                 late = self._read_marker(done_uri)
254             except Exception:  # noqa: BLE001 ? the final check is best-effort too: absence
? failure
255                 LOG.warning(
256                     "cloud-run: final post-timeout marker check failed ? treating as
absent",
257                     exc_info=True,
258                 )
259                 late = None
260             if late is not None:
261                 LOG.warning(
262                     "cloud-run: job=%s exec=%s done-marker found on the final post-timeout
check "
263                     "? rescuing the completed result",
264                     self._job_name,
265                     execution,
266                 )
267                 return late
268             try:
269                 self._client.cancel_execution(execution)
270             except Exception as cancel_exc:  # noqa: BLE001 ? best-effort: never mask the
timeout
271                 cancel_note = f"best-effort cancel FAILED ({cancel_exc})"
272                 LOG.warning(
273                     "cloud-run: best-effort cancel of execution=%s failed: %s",
274                     execution,
275                     cancel_exc,
276                     exc_info=True,
277                 )
278             else:
279                 cancel_note = "cancel requested"
280                 LOG.info("cloud-run: requested cancel of execution=%s", execution)
281             raise PolicyTimeout(
282                 f"{timeout_exc} [cloud-run job={self._job_name} "
283                 f"execution={execution or '<unknown>'}; {cancel_note} ? verify the execution
"
284                 "state in the GCP console]"
285             ) from timeout_exc
286
287         # --- storage-backed completion signal (bucket-poll) ---
288         def _read_marker(self, done_uri: str) -> Optional[dict]:
289             """The done-marker dict if the worker has written it, else None ? poll_until
keeps waiting."""
290             ref = parse_uri(done_uri)
291             if not get_storage(ref.backend, config=self._storage_config).exists(ref):
292                 return None
293             return (
294                 self._read_json(done_uri) or {}
295             )  # empty dict still terminates the poll (present == done)
296
297         def _read_json(self, uri: str) -> dict:
298             # Download to a real local dst FIRST: the cloud backends (gcs/s3) REQUIRE a dst
? a None dst
299             # raises ValueError ? unlike local's identity passthrough. Without an explicit
dst this read
300             # would silently return {} for every gs://|s3:// object (the only backends this
shim targets).
301             try:
302                 ref = parse_uri(uri)
303                 dst = os.path.join(
304                     tempfile.mkdtemp(prefix="cr-read-"), ref.name or "obj.json"
305                 )
306                 local = fetch(uri, dst, config=self._storage_config)
307                 with open(local, "r", encoding="utf-8") as f:
```

```
308                     data: Any = json.load(f)
309                 return data if isinstance(data, dict) else {}
310             except Exception:
311                 LOG.warning("cloud-run: could not read %s", uri, exc_info=True)
312                 return {}
313
```

10. user (2026-07-02T04:56:57.073Z)

```
1       """Per-job execution policy: bounded, observable waits for long/flaky ops
(EXECUTION_POLICY.md P1).
2
3       Turns an unbounded blocking call (the GX010125 Gemini hang) into a bounded one:
4       - ``run_bounded`` ? a HARD per-attempt timeout (``op_timeout_sec``, the hang-killer) +
retry-with-backoff
5         for FAILURES (an exception) *and* HANGS (no return in time). A hung attempt's worker
thread is a daemon,
6         so it is abandoned, not joined ? it never blocks the process.
7       - ``poll_until`` ? a bounded poll loop for an in-progress external op
(``poll_every_n_sec`` cadence,
8         ``max_wait_sec`` deadline).
9
10      **Testability:** ``sleep``/``monotonic`` are injected, so the whole module is unit-
tested with zero real
11      waiting (a fake clock advances instantly).
12
13      **Logging discipline** (so polling is VISIBLE but never NOISY): every poll/retry logs at
**DEBUG**; state
14      transitions (start, resolved, hung, gave-up) log at **INFO/WARNING**. INFO stays clean;
enable DEBUG on the
15      ``execution.policy`` logger to watch the poll trail.
16      """
17
18      from __future__ import annotations
19
20      import logging
21      import threading
22      import time as _time
23      from dataclasses import asdict, dataclass
24      from enum import Enum
25      from typing import Callable, Optional, TypeVar
26
27      T = TypeVar("T")
28
29      #: Dedicated logger so callers can dial polling visibility independently of the app's
root level.
30      LOG = logging.getLogger("execution.policy")
31
32
33      class WaitMode(str, Enum):
34          """How a job treats a long wait. ``str``-valued so it serialises straight to JSON on
the job row."""
35
36          WAIT_AND_RESUME = "wait_and_resume"  # default: bounded wait; over budget ->
reschedule/partial (P2/P3)
37          ABORT = "abort"  # one attempt; on hang/fail, stop immediately
38          BLOCK = "block"  # bounded in-process blocking wait (no reschedule)
39
40
41      class PolicyTimeout(TimeoutError):
42          """An op hung past ``op_timeout_sec`` or never became ready within
``max_wait_sec``."""
43
44
45      @dataclass(frozen=True)
```

```
46      class ExecutionPolicy:
47          """Declarative resilience knobs, attachable per job/op (JSON-serialisable; see
EXECUTION_POLICY.md)."""
48
49          mode: WaitMode = WaitMode.WAIT_AND_RESUME
50          poll_every_n_sec: float = 10.0
51          op_timeout_sec: float = (
52              120.0  # HARD per-attempt deadline ? the hang-killer that was missing
53          )
54          max_wait_sec: float = 1800.0  # total wall budget for a poll loop
55          max_retries: int = 5  # retries (=> max_retries+1 attempts) for failures/hangs
56          backoff_base_sec: float = 3.0  # exponential: backoff_base * 2**attempt
57          on_timeout: str = "reschedule"  # reschedule | partial | fail  (consumed by P2/P3)
58          resumable: bool = True
59
60          def to_dict(self) -> dict:
61              d = asdict(self)
62              d["mode"] = self.mode.value
63              return d
64
65          @classmethod
66          def from_dict(cls, d: dict) -> "ExecutionPolicy":
67              known = {f for f in cls.__dataclass_fields__}  # type: ignore[attr-defined]
68              kw = {k: v for k, v in d.items() if k in known}
69              if "mode" in kw and not isinstance(kw["mode"], WaitMode):
70                  kw["mode"] = WaitMode(kw["mode"])
71              return cls(**kw)
72
73
74      #: The sane default (also what a job gets if it declares no policy).
75      DEFAULT_POLICY = ExecutionPolicy()
76
77
78      def _backoff_sec(policy: ExecutionPolicy, attempt: int) -> float:
79          """Deterministic exponential backoff (no jitter ? reproducible tests)."""
80          return policy.backoff_base_sec * (2**attempt)
81
82
83      def run_bounded(
84          fn: Callable[[], T],
85          policy: ExecutionPolicy = DEFAULT_POLICY,
86          *,
87          label: str = "op",
88          logger: Optional[logging.Logger] = None,
89          sleep: Callable[[float], None] = _time.sleep,
90      ) -> T:
91          """Run ``fn()`` under a hard per-attempt timeout, retrying FAILURES and HANGS with
backoff.
92
93          Returns ``fn``'s value on success. Raises ``PolicyTimeout`` if the final attempt
hung, or re-raises the
94          last exception if the final attempt failed. ``mode=ABORT`` => a single attempt (no
retries).
95          """
96          log = logger or LOG
97          attempts = 1 if policy.mode is WaitMode.ABORT else policy.max_retries + 1
98          last_exc: Optional[BaseException] = None
99          for attempt in range(attempts):
100             box: dict = {}
101             done = threading.Event()
102
103             # box/done bound as defaults (not closed over) ? the thread is started + joined
within this
104             # same iteration, but explicit binding keeps each attempt isolated and silences
B023.
```

```
            def _runner(_box: dict = box, _done: threading.Event = done) -> None:
                try:
                    _box["val"] = fn()
                except (
                    BaseException
                ) as e:  # relay ANY error (incl. timeouts) to the caller thread
                    _box["exc"] = e
                finally:
                    _done.set()

            threading.Thread(target=_runner, name=f"bounded:{label}", daemon=True).start()
            if done.wait(timeout=policy.op_timeout_sec):
                if "exc" in box:
                    last_exc = box["exc"]
                    log.info(
                        "%s: attempt %d/%d failed: %s",
                        label,
                        attempt + 1,
                        attempts,
                        last_exc,
                    )
                else:
                    if attempt:
                        log.info(
                            "%s: succeeded on attempt %d/%d", label, attempt + 1, attempts
                        )
                    return box["val"]  # type: ignore[return-value]
            else:
                last_exc = PolicyTimeout(
                    f"{label} hung > op_timeout_sec={policy.op_timeout_sec}s"
                )
                log.warning(
                    "%s: attempt %d/%d HUNG (>%.0fs) ? abandoning",
                    label,
                    attempt + 1,
                    attempts,
                    policy.op_timeout_sec,
                )
            if attempt < attempts - 1:
                wait = _backoff_sec(policy, attempt)
                log.debug("%s: backing off %.1fs before retry", label, wait)
                sleep(wait)
        assert last_exc is not None
        log.warning("%s: gave up after %d attempt(s): %s", label, attempts, last_exc)
        raise last_exc


    def poll_until(
        check: Callable[[], Optional[T]],
        policy: ExecutionPolicy = DEFAULT_POLICY,
        *,
        label: str = "op",
        logger: Optional[logging.Logger] = None,
        sleep: Callable[[float], None] = _time.sleep,
        monotonic: Callable[[], float] = _time.monotonic,
    ) -> T:
        """Poll ``check()`` every ``poll_every_n_sec`` until it returns a non-``None``
value, or raise
        ``PolicyTimeout`` after ``max_wait_sec``. ``check`` returns ``None`` while the op is
still in progress.

        Polls log at DEBUG (no INFO spam); the wait start + resolution + give-up log at
INFO/WARNING.
        """
        log = logger or LOG
```

```
167         start = monotonic()
168         polls = 0
169         log.info(
170             "%s: waiting (poll_every=%.0fs, max_wait=%.0fs)",
171             label,
172             policy.poll_every_n_sec,
173             policy.max_wait_sec,
174         )
175         while True:
176             result = check()
177             if result is not None:
178                 log.info(
179                     "%s: ready after %d poll(s) / %.0fs", label, polls, monotonic() - start
180                 )
181                 return result
182             elapsed = monotonic() - start
183             if elapsed >= policy.max_wait_sec:
184                 log.warning(
185                     "%s: still not ready after %.0fs (max_wait) ? giving up", label, elapsed
186                 )
187                 raise PolicyTimeout(
188                     f"{label} not ready within max_wait_sec={policy.max_wait_sec}s"
189                 )
190             polls += 1
191             log.debug("%s: poll %d, %.0fs elapsed, not ready", label, polls, elapsed)
192             sleep(policy.poll_every_n_sec)
193
```

11. user (2026-07-02T04:57:49.172Z)

```
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-cloudrun-
cancel\backend\compute\__init__.py:64:    from backend.compute.cloud_run import CloudRunCompute,
GoogleCloudRunJobClient
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-cloudrun-
cancel\backend\compute\__init__.py:69:    client = run_client if run_client is not None else
GoogleCloudRunJobClient()
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-cloudrun-
cancel\backend\compute\cloud_run.py:20:The Cloud Run trigger is an **injected client**
(``CloudRunJobClient``) so the whole shim is unit-tested
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-cloudrun-
cancel\backend\compute\cloud_run.py:21:with a fake (no GCP). The real
``GoogleCloudRunJobClient`` lazy-imports ``google-cloud-run`` and is
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-cloudrun-
cancel\backend\compute\cloud_run.py:54:class CloudRunJobClient(ABC):
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-cloudrun-
cancel\backend\compute\cloud_run.py:80:class GoogleCloudRunJobClient(CloudRunJobClient):
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-cloudrun-
cancel\backend\compute\cloud_run.py:155:        run_client: CloudRunJobClient,
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-cloudrun-
cancel\deploy\cloudrun\README.md:66:- **`GOOGLE_CLOUD_PROJECT` is required** for the real client
? `GoogleCloudRunJobClient.run_job`
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-cloudrun-
cancel\tests\test_api_cloud_dispatch.py:5:GPU-free + cloud-free via a FAKE CloudRunJobClient
that writes report.json + intervals.csv + the
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-cloudrun-
```

```
cancel\tests\test_api_cloud_dispatch.py:21:from backend.compute.cloud_run import
CloudRunJobClient
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-cloudrun-
cancel\tests\test_api_cloud_dispatch.py:39:class _FakeCloudClient(CloudRunJobClient):
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-cloudrun-
cancel\tests\test_compute_backend.py:3:All GPU-free + cloud-free: the Cloud Run trigger is a
FAKE ``CloudRunJobClient`` that simulates the
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-cloudrun-
cancel\tests\test_compute_backend.py:16:from backend.compute.cloud_run import CloudRunCompute,
CloudRunJobClient
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-cloudrun-
cancel\tests\test_compute_backend.py:24:class _FakeRunClient(CloudRunJobClient):
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-cloudrun-
cancel\tests\test_compute_backend.py:72:class _NoopRunClient(CloudRunJobClient):
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-cloudrun-
cancel\tests\test_compute_backend.py:90:class _NeverDoneClient(CloudRunJobClient):
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-cloudrun-
cancel\tests\test_compute_backend.py:141:    """The real GoogleCloudRunJobClient must fail
loudly when GOOGLE_CLOUD_PROJECT is unset ?
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-cloudrun-
cancel\tests\test_compute_backend.py:145:    from backend.compute.cloud_run import
GoogleCloudRunJobClient
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-cloudrun-
cancel\tests\test_compute_backend.py:147:    client = GoogleCloudRunJobClient()
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-cloudrun-
cancel\tests\test_compute_backend.py:160:    from backend.compute.cloud_run import
GoogleCloudRunJobClient
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-cloudrun-
cancel\tests\test_compute_backend.py:162:    client = GoogleCloudRunJobClient()
```

12. user (2026-07-02T04:57:49.697Z)

```
1061-        segmenter=seg_name,
1062-        config_overrides=tuple(overrides),
1063-    )
1064-    try:
1065-        result = compute.run_infer(
1066-            spec
1067-        )  # dispatch the Cloud Run Job + bucket-poll to completion
1068-    except Exception as e:  # noqa: BLE001 ? any dispatch/poll error is a clean job
failure, not a 500
1069:        logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
1070:        return _failed(f"cloud dispatch error: {e}", True)
1071-    if result.returncode != 0:
1072-        return _failed(f"cloud job failed (returncode {result.returncode}).", True)
1073-    if result.video_id and result.video_id != video_id:
1074-        # The worker re-hashes the bucket bytes; a divergence means the input changed
between this
1075-        # box's hash and the worker's ? ingest under the LOCAL id but flag it so it's
diagnosable.
1076-        logger.warning(
1077-            "[API] cloud video_id %s != local %s ? input bytes differ between the API hash
"
1078-            "and the worker hash; ingesting under the local id.",
```

13. assistant (2026-07-02T04:57:57.216Z)

```
All implementers are covered. Now let me run the verification gates in the worktree ? check CI
config first for the coverage floor.
```

14. user (2026-07-02T04:58:00.968Z)

```
ci.yml
nightly.yml
.github/workflows/ci.yml:135:        # (pytest, pytest-cov, httpx for the fastapi TestClient,
ruff, mypy).
.github/workflows/ci.yml:138:      # pytest.importorskip ? run_all_tests.py prints which. The
coverage floor
.github/workflows/ci.yml:142:      # with obsreport: 73%). Ratchet it UP as coverage grows;
never down
.github/workflows/ci.yml:144:      - name: Run full suite (with coverage floor)
.github/workflows/ci.yml:145:        run: python run_all_tests.py --cov=backend --cov=training
--cov-report=term --cov-fail-under=70
.github/workflows/ci.yml:171:        # de-attribution paths get cross-OS coverage and surface
any win/mac float drift.
.github/workflows/nightly.yml:17:#   - golden-fixtures runs single-OS here (ci.yml::golden-
crossos already covers
.github/workflows/nightly.yml:22:# Lane set + coverage floor are still copied literals, not
single-sourced ? see
.github/workflows/nightly.yml:107:  # Full offline suite + coverage floor ? mirrors
ci.yml::full-suite. CPU-only
.github/workflows/nightly.yml:138:      - name: Run full suite (with coverage floor)
.github/workflows/nightly.yml:139:        run: python run_all_tests.py --cov=backend
--cov=training --cov-report=term --cov-fail-under=70
.github/workflows/nightly.yml:144:  # ci.yml::golden-crossos (single-OS here; ci.yml already
covers cross-OS on
.github/workflows/nightly.yml:253:              'One of ruff / leak-scan / mypy / import-smoke /
full-suite (cov>=70) /',
```

15. user (2026-07-02T04:58:30.262Z)

```
tests/test_api_cloud_dispatch.py::test_local_override_survives_api_boundary
tests/test_api_cloud_dispatch.py::test_cloud_override_survives_api_boundary_on_local_server
tests/test_api_cloud_dispatch.py::test_full_api_process_job_result_carries_cloud_provenance
  C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end
user credentials from Google Cloud SDK without a quota project. You might receive a "quota
exceeded" or "API not enabled" error. See the following page for troubleshooting:
https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
    warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
42 passed, 4 warnings in 25.72s
```

16. assistant (2026-07-02T04:58:30.949Z)

```
You've hit your session limit · resets 1:10pm (Asia/Calcutta)
```